K-Way Merge and Pairwise Merge in C

Program for three sorted transaction lists, including Min Heap states, merged outputs, and operation counts.

Input Lists

```
L1 = {10, 30, 50, 70}\nL2 = {20, 40, 60, 80}\nL3 = {15, 35, 55, 75}
```

C Program

```
#include <stdio.h>

#define K 3
#define M 4
#define N (K * M)

int comparisons = 0;

typedef struct {
    int value;
    int list;
    int index;
} Node;

void swap(Node *a, Node *b) {
    Node temp = *a;
    *a = *b;
    *b = temp;
}

void heapify(Node heap[], int size, int i) {
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < size) {
        comparisons++;
        if (heap[left].value < heap[smallest].value)
            smallest = left;
    }

    if (right < size) {
        comparisons++;
        if (heap[right].value < heap[smallest].value)
            smallest = right;
    }

    if (smallest != i) {
        swap(&heap[i], &heap[smallest]);
        heapify(heap, size, smallest);
    }
}

void kWayMerge(int lists[K][M]) {
    Node heap[K];
    int size = K;
    int output[N], count = 0;
    int i;

    comparisons = 0;

    for (i = 0; i < K; i++) {
        heap[i].value = lists[i][0];
        heap[i].list = i;
        heap[i].index = 0;
    }

    /* Build the initial Min Heap */
    for (i = size / 2 - 1; i >= 0; i--)
        heapify(heap, size, i);

    printf("\nInitial Heap: ");
    for (i = 0; i < size; i++)
        printf("%d ", heap[i].value);

    while (size > 0) {
        Node root = heap[0];
        output[count++] = root.value;

        int next = root.index + 1;

        if (next < M) {
            heap[0].value = lists[root.list][next];
            heap[0].index = next;
```

```
        } else {
            heap[0] = heap[size - 1];
            size--;
        }

        if (size > 0)
            heapify(heap, size, 0);

        printf("\nHeap: ");
        for (i = 0; i < size; i++)
            printf("%d ", heap[i].value);
    }

    printf("\n\nK-way Merge Output: ");
    for (i = 0; i < count; i++)
        printf("%d ", output[i]);

    printf("\nHeap element comparisons: %d\n", comparisons);
}

int merge(int a[], int n1, int b[], int n2, int result[]) {
    int i = 0, j = 0, p = 0;

    while (i < n1 && j < n2) {
        comparisons++;

        if (a[i] <= b[j])
            result[p++] = a[i++];
        else
            result[p++] = b[j++];
    }

    while (i < n1)
        result[p++] = a[i++];

    while (j < n2)
        result[p++] = b[j++];

    return p;
}

void pairwiseMerge(int lists[K][M]) {
    int first[8], result[N];
    int i, n;

    comparisons = 0;

    n = merge(lists[0], M, lists[1], M, first);
    n = merge(first, n, lists[2], M, result);

    printf("\nPairwise Merge Output: ");
    for (i = 0; i < n; i++)
        printf("%d ", result[i]);

    printf("\nPairwise element comparisons: %d\n", comparisons);
}

int main(void) {
    int lists[K][M] = {
        {10, 30, 50, 70},
        {20, 40, 60, 80},
        {15, 35, 55, 75}
    };

    printf("Input Lists:\n");
    for (int i = 0; i < K; i++) {
        printf("L%d: ", i + 1);
        for (int j = 0; j < M; j++)
            printf("%d ", lists[i][j]);
        printf("\n");
    }

    printf("\n--- K-Way Merge Using Min Heap ---\n");
    kWayMerge(lists);

    printf("\n--- Pairwise Merge ---\n");
    pairwiseMerge(lists);

    return 0;
}
```

Expected Output

The heap comparison count is printed by the program at runtime and may depend on implementation details.

```
Input Lists:
L1: 10 30 50 70
L2: 20 40 60 80
L3: 15 35 55 75

--- K-Way Merge Using Min Heap ---
Initial Heap: 10 20 15
Heap: 15 20 30
Heap: 20 30 35
Heap: 30 35 40
Heap: 35 40 50
Heap: 40 50 55
Heap: 50 55 60
Heap: 55 60 70
Heap: 60 70 75
Heap: 70 75 80
Heap: 75 80
Heap: 80
Heap:

K-way Merge Output:
10 15 20 30 35 40 50 55 60 70 75 80

--- Pairwise Merge ---
Pairwise Merge Output:
10 15 20 30 35 40 50 55 60 70 75 80
Pairwise element comparisons: 18
```

Final Sorted List

```
10 15 20 30 35 40 50 55 60 70 75 80
```

Complexity Summary

```
Min Heap k-way merge: O(n log k) time; O(k) heap space, excluding output.\nPairwise merge: O(n log k) with balanced merging; O(
```